2º DESARROLLO DE APLICACIONES WEB

CyberPrompt: Hackeando el Contexto de la IA

Fundamentos de la IA Generativa

Alumno: César Valverde Pardo

Curso académico: 2026–2027

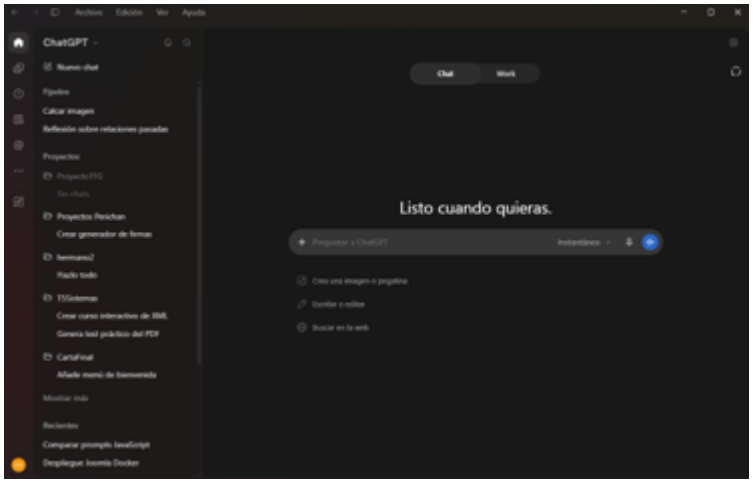
Índice

1. Anatomía de un prompt 4
1.1. Abrimos ChatGPT 4
1.2. Prompt A: Básico 4
1.3. Prompt B: Estructurado 4
1.4. Comparación de las respuestas: 5
1.5. ¿Cuál es más clara? 5
1.6. ¿Cuál se ajusta mejor al objetivo educativo? 5
1.7. Identificación de las partes de cada prompt 6
1.8. Mi versión optimizada del prompt 6
1.9. Conclusión 7
2. Refinamiento de un prompt 7
2.1. Abre ChatGPT 7
2.2. Prompt inicial 7
2.3 Observación de la respuesta 8
2.4 Mejora de Prompt (x3) 8
2.5 Tabla de iteraciones 10
2.6 Explicación del proceso 10
2.7 Conclusión 10
3. Diseño de prompt con buenas prácticas 11
3.1 Creación de prompt 11
3.2 Observaciones 12
4. Aplicar Chain-of-Thought 13
4.1 Mandar Código + Prompt a ChatGPT 13
4.2 Respuesta de ChatGPT 13
4.3 Código corregido: 13
4.4 Evaluación de la respuesta 14
4.5 Conclusiones 14
5. Self-Consistency en generación de código 15
5.1 Prompt: 15

5.2 Tabla Comparativa 16
5.3 Elección de la mejor versión 16
6. Crear un prompt ReAct 17
6.1 Prompt inicial 17
6.2 Respuesta 17
Fase 1 (Análisis): 17
Fase 2: Act (generación de código) 17
6.3 Tabla comparativa de la comprobación entre fases 18
6.4 Prompt mejorado 19
6.5 Observaciones del proceso 19
7. Arbol de decisiones (Tree of thoughts) 20
7.1 Prompt 20
7.2 Respuesta: 20
7.3 Arbol de decisiones 22
7.4 Evaluación de la respuesta 22
7.5 Recomendación Personal 22
8. Refactorización Guiada 23
8.1 Prompt 23
8.2 Respuesta 23
8.3 Evaluación del código 23
8.4 Prompt final mejorado 23
8.5 Explicación de las mejoras introducidas 24
8.6 Explicación del resultado 24

1. Anatomía de un prompt

1.1. Abrimos ChatGPT



1.2. Prompt A: Básico

Prompt: Explícame JavaScript

Respuesta:

JavaScript es un lenguaje de programación utilizado principalmente en el desarrollo web. Permite crear páginas interactivas y dinámicas, como formularios, botones, animaciones y menús desplegables.

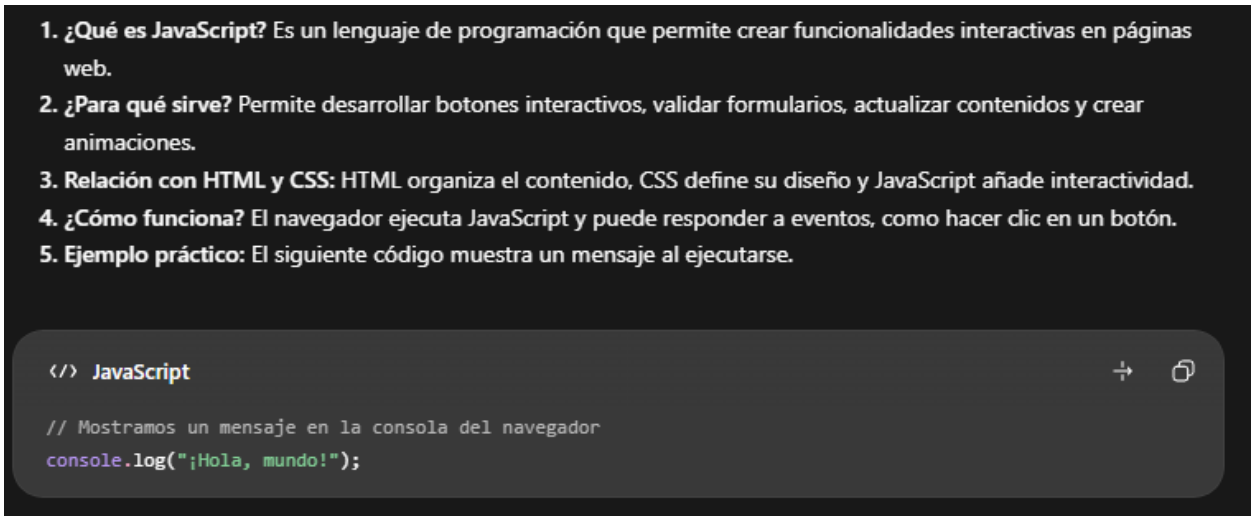
Se utiliza junto con HTML, que estructura el contenido, y CSS, que define su apariencia. JavaScript permite añadir comportamientos y responder a las acciones del usuario.

También puede utilizarse fuera del navegador, por ejemplo, en servidores mediante Node.js.

1.3. Prompt B: Estructurado

Prompt: Eres un profesor de desarrollo web. Explica en 5 puntos claros qué es JavaScript, qué papel tiene en una página web y muestra un ejemplo simple de código con comentarios

Respuesta:



1.4. Comparación de las respuestas:

Criterio	Prompt A	Prompt B
Claridad	Explicación general	Explicación organizada
Estructura	Texto libre	Cinco puntos definidos
Objetivo educativo	Introducción básica	Enseñanza guiada
Ejemplo de código	No se exige	Se exige
Adaptación al estudiante	No especificada	Rol de profesor
Precisión de instrucciones	Baja	Alta

1.5. ¿Cuál es más clara?

La respuesta más clara es la B porque divide la información en cinco puntos, sigue un orden lógico y proporciona un ejemplo práctico

1.6. ¿Cuál se ajusta mejor al objetivo educativo?

Prompt B se ajusta mejor, ya que establece un rol docente, determina los contenidos que deben explicarse y exige una estructura concreta.

Prompt A también es válido para obtener una explicación rápida, pero deja a la IA decidir la extensión, el nivel y el formato de la respuesta.

Esta comparación es ilustrativa, un prompt más detallado suele dar mayor control sobre la respuesta, pero no garantiza automáticamente que sea más correcta

1.7. Identificación de las partes de cada prompt

Elemento	Prompt A	Prompt B
Rol	No definido	Profesor de desarrollo web
Instrucción	Explicar JavaScript	Explicar qué es y su función
Contexto	No definido	Enseñanza de desarrollo web
Formato	Libre	Cinco puntos claros
Ejemplo	No solicitado	Código con comentarios
Público objetivo	No especificado	Implícitamente educativo

1.8. Mi versión optimizada del prompt

Actúa como un profesor de desarrollo web especializado en enseñar a estudiantes principiantes que no tienen conocimientos previos de programación.

Explica qué es JavaScript utilizando un lenguaje sencillo, claro y educativo.

Organiza la explicación en **5 puntos numerados**:

1. Qué es JavaScript.
2. Para qué sirve en una página web.
3. Cuáles son las diferencias entre HTML, CSS y JavaScript.
4. Cómo funciona JavaScript en el navegador.
5. Un ejemplo práctico de JavaScript aplicado a una página web.

Incluye un ejemplo sencillo de código que permita cambiar el texto de una página al pulsar un botón. Añade comentarios que expliquen las instrucciones más importantes.

Evita tecnicismos innecesarios, utiliza ejemplos cotidianos y termina con una pregunta de repaso para comprobar que el estudiante ha comprendido la explicación.

1.9. Conclusión

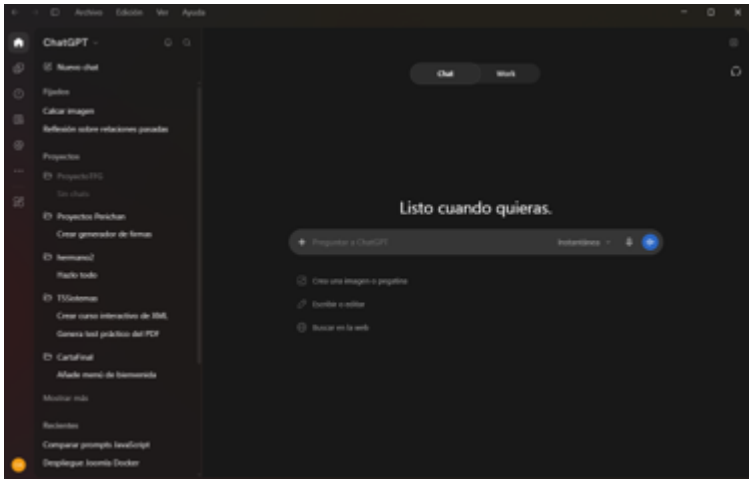
El prompt B es más adecuado que el A para un objetivo educativo porque ofrece instrucciones específicas y establece cómo debe organizarse la información.

La versión optimizada mejora el prompt B al definir el nivel del estudiante, solicitar un ejemplo interactivo y añadir una pregunta final para comprobar el aprendizaje.

Conclusión principal: cuanto más claro y específico sea un prompt respecto al objetivo, el público y el formato, mayor control tendremos sobre la utilidad de la respuesta generada por la IA

2. Refinamiento de un prompt

2.1. Abre ChatGPT



2.2. Prompt inicial

Prompt: Crea una función en JavaScript que convierta una cadena en formato kebab-case
Respuesta:



2.3 Observación de la respuesta

La función convierte las letras a minúsculas y sustituye los espacios por guiones. Sin embargo, no valida las entradas, no contiene comentarios ni muestra ejemplos de uso

2.4 Mejora de Prompt (x3)

Prompt 1: Crea una función en JavaScript que convierta una cadena en formato kebab-case. Añade una validación para comprobar que el valor introducido sea una cadena de texto y muestra un error si no lo es

Respuesta 1:

```javascript
function kebabCase(texto) {
  if (typeof texto !== "string") {
    throw new TypeError("Debes introducir un texto");
  }

  return texto.toLowerCase().trim().replace(/\s+/g, "-");
}
```

Prompt 2: Crea una función en JavaScript que convierta una cadena en formato kebab-case, valide que la entrada sea un texto e incluya comentarios que expliquen cada paso del código

Respuesta 2:

```javascript
function kebabCase(texto) {
  // Comprobar que la entrada sea texto
  if (typeof texto !== "string") {
    throw new TypeError("Debes introducir un texto");
  }

  // Eliminar espacios al inicio y al final
  texto = texto.trim();

  // Convertir el texto a minúsculas
  texto = texto.toLowerCase();

  // Sustituir espacios por guiones
  texto = texto.replace(/\s+/g, "-");

  // Devolver el resultado
  return texto;
}
```

Prompt 3: Actúa como un profesor de JavaScript. Crea una función que convierta una cadena en formato kebab-case, valide que la entrada sea un texto, incluya comentarios explicativos y muestre tres ejemplos de uso con sus resultados esperados

Respuesta 3:

```
function kebabCase(texto) {
  // Validar que la entrada sea una cadena
  if (typeof texto !== "string") {
    throw new TypeError("Debes introducir un texto");
  }

  // Eliminar espacios innecesarios
  texto = texto.trim();

  // Convertir a minúsculas
  texto = texto.toLowerCase();

  // Sustituir espacios por guiones
  texto = texto.replace(/\s+/g, "-");

  // Devolver el texto convertido
  return texto;
}

// Ejemplo 1
console.log(kebabCase("Hola Mundo"));
// Resultado: hola-mundo

// Ejemplo 2
console.log(kebabCase("Aprendiendo JavaScript"));
// Resultado: aprendiendo-javascript

// Ejemplo 3
console.log(kebabCase("Mi Primera Página Web"));
// Resultado: mi-primera-página-web
```

2.5 Tabla de iteraciones

Iteración	Prompt usado	Cambios realizados	Mejora observada
Inicial	Crea una función en JavaScript que convierta una cadena en formato kebab-case.	Ninguno.	Genera una función básica.
1	Crea una función que convierta texto a kebab-case y valide que la entrada sea una cadena.	Añadir validación.	Evita procesar entradas de tipos incorrectos.
2	Crea una función que convierta texto a kebab-case, valide las entradas e incluya comentarios explicativos.	Añadir comentarios.	Facilita la comprensión del código.
3	Actúa como profesor de JavaScript. Crea una función que convierta texto a kebab-case, valide las entradas, incluya comentarios y muestre tres ejemplos.	Añadir rol y ejemplos de uso.	Permite comprender y comprobar mejor el funcionamiento.

2.6 Explicación del proceso

Al principio, el prompt era muy sencillo y solo pedía convertir una cadena a kebab-case.

En la primera mejora se añadió una validación para evitar errores cuando se introducen datos que no son texto.

En la segunda mejora se solicitaron comentarios dentro del código, lo que permite entender mejor cada instrucción.

Por último, en la tercera mejora se añadieron ejemplos de uso y se asignó a la IA el rol de profesor de JavaScript, consiguiendo una respuesta más orientada al aprendizaje

2.7 Conclusión

Con esta actividad he comprobado como el refinamiento de prompts permite mejorar progresivamente el código generado por una IA, al inicio era un prompt sencillo, mientras que el prompt final, consiguió una función para validar entradas, comentarios y ejemplos prácticos

3. Diseño de prompt con buenas prácticas

3.1 Creación de prompt

El prompt debe: Generar función JS que valide un formulario HTML (nombre, email, contraseña), que comente cada línea del código y que explique al final cómo integrarlo en una página web

Prompt: Crea una función en JavaScript que valide un formulario HTML con tres campos: nombre, email y contraseña. Añade comentarios en cada línea del código y explica al final cómo integrarlo en una página web.

Respuesta:



```javascript
function validarFormulario() {
  // Obtener los valores del formulario
  let nombre = document.getElementById("nombre").value;
  let email = document.getElementById("email").value;
  let password = document.getElementById("password").value;

  // Comprobar que los campos no estén vacíos
  if (nombre === "" || email === "" || password === "") {
    alert("Todos los campos son obligatorios");
    return false;
  }

  return true;
}
```

Explicación de cómo integrarlo en una página web según la respuesta de ChatGPT:

1. Abrir Visual Studio Code o cualquier editor de texto.
2. Crear un archivo llamado index.html.
3. Copiar el código HTML y JavaScript anterior.
4. Guardar el archivo.
5. Abrirlo en un navegador, como Chrome o Firefox.
6. Introducir datos en el formulario y pulsar el botón Enviar.

Si algún dato es incorrecto, aparecerá un mensaje de error. Si todos los datos cumplen las condiciones, aparecerá un mensaje indicando que el formulario se ha validado correctamente.

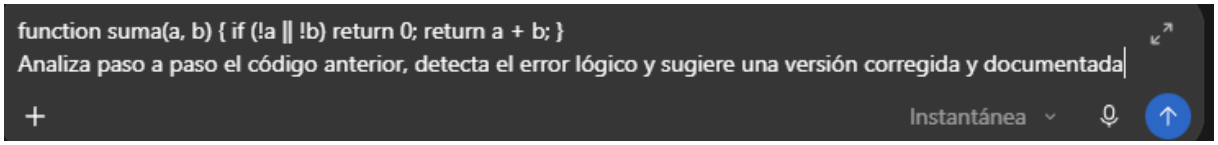
La validación se realiza en el navegador y no envía los datos a un servidor. En una aplicación real también sería necesario validar los datos en el servidor.

3.2 Observaciones

Aspecto	Prompt inicial	Prompt final
Rol	No especificado	Desarrollador y profesor
Validación del nombre	Solo comprueba si está vacío	Comprueba el nombre sin espacios exteriores
Validación del email	No comprueba el formato	Verifica el formato
Contraseña	Solo comprueba si está vacía	Exige mínimo 8 caracteres
Comentarios	Incompletos en la respuesta de ejemplo	Comentarios explicativos en cada instrucción JavaScript
Integración	No incluye HTML	Incluye una página HTML completa
Resultado	Código básico	Ejemplo funcional y documentado

4. Aplicar Chain-of-Thought

4.1 Mandar Código + Prompt a ChatGPT



4.2 Respuesta de ChatGPT

Paso 1. Analizar la función

La función `suma(a, b)` recibe dos parámetros y pretende devolver su suma.

Paso 2. Detectar el problema

La condición `if (!a || !b)` comprueba si alguno de los valores se evalúa como falso.

En JavaScript, el número `0` se considera un valor *falsy* (falso en un contexto booleano).

Paso 3. Identificar el error lógico

Al ejecutar `suma(0, 5)`, JavaScript interpreta `!0` como `true`, por lo que se cumple la condición y la función devuelve `0` sin realizar la suma.

Paso 4. Proponer una solución

Se debe eliminar esa comprobación y sustituirla por una validación que verifique que ambos parámetros son números válidos.

Paso 5. Comprobar el resultado

Con el código corregido, `suma(0, 5)` devuelve `5`, que es el resultado matemático correcto.

4.3 Código corregido:

```
function suma(a, b) {
  // Comprobar que ambos valores sean números válidos
  if (
    typeof a !== "number" ||
    typeof b !== "number" ||
    !Number.isFinite(a) ||
    !Number.isFinite(b)
  ) {
    // Mostrar un error si algún valor no es válido
    throw new TypeError("Los valores deben ser números válidos");
  }
```

```
  // Devolver la suma de los dos números
  return a + b;
}

// Ejemplo 1: Suma con cero
console.log(suma(0, 5)); // Resultado: 5

// Ejemplo 2: Suma normal
console.log(suma(3, 7)); // Resultado: 10

// Ejemplo 3: Suma con números negativos
console.log(suma(-2, 4)); // Resultado: 2
```

4.4 Evaluación de la respuesta

Criterio	Evaluación
¿Detecta el error lógico?	Sí, identifica que el problema está en if (!a || !b).
¿Explica por qué ocurre?	Sí, porque 0 se evalúa como falso en JavaScript.
¿Propone una solución funcional?	Sí, elimina la condición incorrecta y valida los parámetros.
¿Incluye comentarios?	Sí, explica las partes principales del código.
¿Muestra resultados correctos?	Sí, suma(0, 5) devuelve 5.

4.5 Conclusiones

Aprendí a usar un prompt que solicita análisis paso a paso para deteectar errores lógicos en JS, además, el problema del código original era que consideraba el número 0 como un valor falso y devolvía 0 sin realizar la operación, y, por el análisis, indentifiqué el error y se obtuvo una versión corregida que permite sumar correctamente números positivos, negativos y el cero

5. Self-Consistency en generación de código

5.1 Prompt:

Actúa como un experto en JavaScript. Genera tres versiones diferentes de una función que invierta una cadena de texto. Utiliza distintas formas de resolver el problema, incluye comentarios explicativos y muestra ejemplos de uso. Después, compara las soluciones según su claridad, eficiencia y utilización de funciones nativas de JavaScript.

Versión 1:

```javascript
function invertirTexto(texto) {
  // Convertir el texto en un array,
  // invertirlo y unirlo nuevamente
  return texto.split("").reverse().join("");
}

console.log(invertirTexto("Hola"));
// Resultado: aloH
```

Version 2:

```javascript
function invertirTexto(texto) {
  // Crear una cadena vacía
  let resultado = "";

  // Recorrer el texto desde el final
  for (let i = texto.length - 1; i >= 0; i--) {
    resultado += texto[i];
  }

  // Devolver el texto invertido
  return resultado;
}

console.log(invertirTexto("Hola"));
// Resultado: aloH
```

Version 3:

```javascript
function invertirTexto(texto) {
  // Convertir el texto en un array
  const caracteres = Array.from(texto);

  // Invertir el orden de los caracteres
  caracteres.reverse();

  // Unir los caracteres en una cadena
  return caracteres.join("");
}

console.log(invertirTexto("Hola"));
// Resultado: aloH
```

5.2 Tabla Comparativa

Criterio	Versión 1	Versión 2	Versión 3
Claridad	Alta	Media	Alta
Funciones nativas	split, reverse, join	No utiliza	Array.from, reverse, join
Documentación	Básica	Buena	Buena
Eficiencia	Buena	Buena	Buena
Líneas de código	Pocas	Más líneas	Intermedia
Facilidad de mantenimiento	Alta	Media	Alta
Tratamiento de Unicode	Limitado	Limitado	Mejor

5.3 Elección de la mejor versión

He elegido la versión 3, porque ofrece un buen equilibrio entre claridad, documentación y uso de funciones nativas.

Sus principales ventajas son que divide el proceso en pasos fáciles de entender, utiliza funciones propias de JavaScript y maneja correctamente caracteres representados mediante pares sustitutos, como muchos emojis.

La versión 1 es más corta y también sería adecuada para textos sencillos, pero la 3 es más didáctica y la prefiero

6. Crear un prompt ReAct

6.1 Prompt inicial

Actúa como un experto en seguridad informática y programación JavaScript.

Primero, explica qué condiciones debe cumplir una contraseña para considerarse segura (fase Reason).

Después, genera una función llamada validarPassword() en JavaScript que compruebe esas condiciones (fase Act).

Añade comentarios explicativos al código y muestra ejemplos de contraseñas válidas e inválidas.

6.2 Respuesta

Fase 1 (Análisis):

Para considerar una contraseña suficientemente fuerte, se pueden establecer las siguientes condiciones:

1. Tener al menos 12 caracteres.
2. Contener al menos una letra mayúscula.
3. Contener al menos una letra minúscula.
4. Incluir al menos un número.
5. Contener al menos un carácter especial, como @, # o !.

Estas condiciones sirven como criterios para el ejercicio. En aplicaciones reales, también es importante permitir contraseñas largas, comprobar que no estén comprometidas y evitar reglas de composición innecesariamente restrictivas

Fase 2: Act (generación de código)

```
function validarPassword(password) {
  // Comprobar que la entrada sea texto
  if (typeof password !== "string") {
    return false;
  }

  // Comprobar que tenga al menos 12 caracteres
  if (password.length < 12) {
    return false;
  }
```

```
  // Comprobar que contenga una mayúscula
  if (!/[A-Z]/.test(password)) {
    return false;
  }

  // Comprobar que contenga una minúscula
  if (!/[a-z]/.test(password)) {
    return false;
  }

  // Comprobar que contenga un número
  if (!/[0-9]/.test(password)) {
    return false;
  }

  // Comprobar que contenga un carácter especial
  if (!/[^A-Za-z0-9]/.test(password)) {
    return false;
  }

  // Si cumple todas las condiciones, es válida
  return true;
}

// Ejemplos de uso
console.log(validarPassword("Hola123"));
// Resultado: false

console.log(validarPassword("Segura2026@Clave"));
// Resultado: true

console.log(validarPassword("contrasenamuy larga"));
// Resultado: false

console.log(validarPassword("MiClaveSegura2026!"));
// Resultado: true
```

6.3 Tabla comparativa de la comprobación entre fases

Fase	Requisito	¿Se cumple?
Reason	Explicar las condiciones de una contraseña fuerte.	Sí
Reason	Definir criterios claros de validación.	Sí

Act	Crear la función validarPassword().	Sí
Act	Aplicar las condiciones definidas.	Sí
Act	Incluir comentarios explicativos.	Sí
Act	Mostrar ejemplos de uso.	Sí

6.4 Prompt mejorado

Actúa como un experto en seguridad informática y JavaScript.

Fase Reason: Explica brevemente qué requisitos debe cumplir una contraseña segura, teniendo en cuenta su longitud, la facilidad con la que podría adivinarse y las buenas prácticas actuales de seguridad.

Fase Act: Genera una función JavaScript llamada validarPassword() que:

1. Compruebe que la entrada sea una cadena de texto.
2. Exija un mínimo de 12 caracteres para este ejercicio.
3. Permita espacios y caracteres especiales.
4. Devuelva un objeto indicando si la contraseña es válida y los errores encontrados.
5. Incluya comentarios explicativos.
6. Muestre al menos cuatro ejemplos de uso, incluyendo casos válidos e inválidos.

Finalmente, explica cómo funciona la función y qué limitaciones tiene para utilizarse en una aplicación real.

6.5 Observaciones del proceso

El prompt inicial organiza la respuesta en 2 fases, la primera ayudó a establecer condiciones que debía comprobar la contraseña, en la segunda, las condiciones las volvió en JS, la mejora permite solicitar una valicadicón flexible y mensajes específicos para identificar los errores

7. Arbol de decisiones (Tree of thoughts)

7.1 Prompt

Actúa como un desarrollador web experto. Evalúa tres posibles formas de almacenar datos de usuario en una aplicación web: LocalStorage, IndexedDB y una base de datos remota.

Explica las ventajas e inconvenientes de cada opción, compara su facilidad de implementación, capacidad de almacenamiento, seguridad y funcionamiento sin conexión.

Finalmente, recomienda la mejor alternativa para un proyecto pequeño y justifica tu elección.

7.2 Respuesta:

Opción 1: LocalStorage

LocalStorage permite almacenar información directamente en el navegador mediante pares de clave y valor.

Ventajas:

- Es fácil de utilizar con JavaScript.
- No necesita servidor ni configuración compleja.
- Los datos permanecen guardados después de cerrar el navegador.

Inconvenientes:

- Tiene una capacidad limitada, normalmente alrededor de 5 MiB por origen.
- No permite consultas complejas.
- Los datos no se sincronizan automáticamente entre dispositivos.
- No es adecuado para almacenar contraseñas ni información sensible.

Opción 2: IndexedDB

IndexedDB es una base de datos integrada en los navegadores que permite almacenar información estructurada.

Ventajas:

- Permite guardar cantidades de datos mayores que LocalStorage.
- Admite búsquedas mediante índices.
- Puede funcionar sin conexión a Internet.
- Es adecuada para aplicaciones con muchos registros.

Inconvenientes:

- Su implementación es más compleja.
- Requiere aprender a trabajar con operaciones asíncronas.
- Los datos permanecen asociados al navegador y al dispositivo, salvo que se implemente sincronización.

Opción 3: Base de datos remota

Una base de datos remota almacena la información en un servidor y permite acceder a ella a través de una aplicación backend o una API.

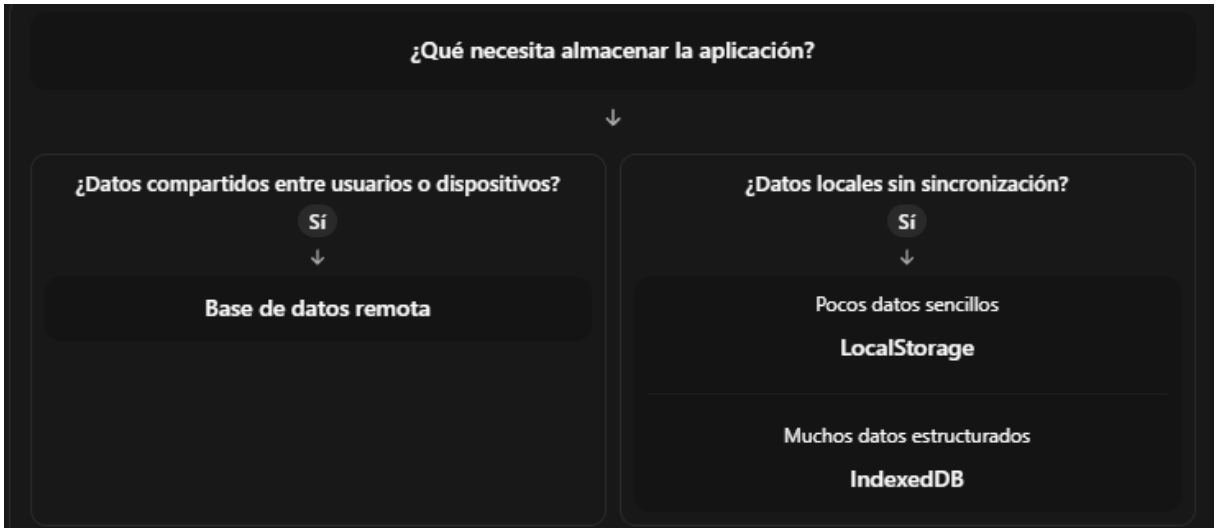
Ventajas:

- Permite sincronizar datos entre dispositivos.
- Facilita la gestión de múltiples usuarios.
- Ofrece opciones de autenticación, copias de seguridad y control de acceso.
- Es adecuada para proyectos que necesitan crecer.

Inconvenientes:

- Requiere más configuración y mantenimiento.
- Puede generar costes de alojamiento.
- Normalmente necesita conexión a Internet.
- Exige implementar correctamente la seguridad del servidor y de la API

7.3 Arbol de decisiones



7.4 Evaluación de la respuesta

Pregunta	Evaluación
¿Explica las tres opciones?	Sí, describe LocalStorage, IndexedDB y una base de datos remota.
¿Presenta ventajas e inconvenientes?	Sí, identifica los principales puntos fuertes y limitaciones.
¿Compara las alternativas?	Sí, mediante una tabla y un árbol de decisiones.
¿Justifica la elección final?	Sí, recomienda LocalStorage para proyectos pequeños con datos simples y locales.
¿Considera otras necesidades?	Sí, contempla aplicaciones con muchos datos o varios usuarios.

7.5 Recomendación Personal

Después de comparar las alternativas, pienso que LocalStorage es la más sencilla para un proyecto pequeño siempre que los datos sean simples y no sensibles, IndexedDB da más cosas pero necesita una implementación más compleja, por otro lado, una BBDD remota es más apropiada para apps con usuarios y datos registrados y sincronizados

8. Refactorización Guiada

8.1 Prompt

Eres un desarrollador web senior. Refactoriza el siguiente código para que acepte un número variable de argumentos, use programación moderna de JavaScript y comente cada paso. Después explica las mejoras introducidas.

function suma(a, b, c, d) {

return a + b + c + d;

}

8.2 Respuesta:

```
function suma(...numeros) {
  // Sumar todos los números recibidos
  return numeros.reduce((total, numero) => total + numero, 0);
}

// Ejemplo de uso
console.log(suma(1, 2, 3, 4)); // 10
console.log(suma(5, 10));      // 15
```

8.3 Evaluación del código

El código utiliza el operador rest (...) y el método reduce(), que permiten sumar una cantidad variable de argumentos.

Sin embargo, no comprueba que los valores recibidos sean números válidos. Por ejemplo, si introducimos una cadena de texto, JavaScript podría concatenar valores en lugar de sumarlos.

Por eso, es conveniente mejorar el prompt.

8.4 Prompt final mejorado

Actúa como un desarrollador web senior especializado en JavaScript moderno.

Refactoriza la siguiente función para que:

1. Acepte un número variable de argumentos utilizando el operador rest (...).
2. Utilice el método reduce() para realizar la suma.
3. Compruebe que todos los argumentos sean números válidos.
4. Muestre un error si se introduce un dato incorrecto.

5. Incluya comentarios explicativos en cada paso.
6. Muestre ejemplos de uso con diferentes cantidades de argumentos.

Finalmente, explica las mejoras realizadas respecto al código original utilizando un lenguaje sencillo.

Código original:

```
function suma(a, b, c, d) {
  return a + b + c + d;
}
```

(NO PEGO LA RESPUESTA, ES MUY LARGA)

8.5 Explicación de las mejoras introducidas

Aspecto	Código original	Código refactorizado
Número de argumentos	Cuatro parámetros definidos	Cantidad variable
Sintaxis	Tradicional	Operador rest y función flecha
Método de suma	Operador + repetido	Método reduce()
Validación	No tiene	Comprueba los tipos y valores
Errores	No se controlan	Genera un error para datos inválidos
Comentarios	No tiene	Incluye explicaciones
Flexibilidad	Limitada	Mayor

8.6 Explicación del resultado

El código final cumple los requisitos establecidos en el prompt mejorado.

La función acepta diferentes cantidades de números, utiliza características modernas de JavaScript y comprueba que los datos sean válidos.

Además, incluye comentarios y ejemplos que facilitan su comprensión